ROKOCO

PLA.NET

Konzept der Schnittstellenbeschreibungen für den Bereich Schaden- / Unfall

Version 11.0
Grünwald 28.11.2025
© 2025 ROKOCO GmbH

Versionshistorie

Version 7.2	Erstversion: Einleitung; Dokumentation Schnittstellen Import PLA.NET	Nicole Erifiu Marius Reitz	03.01.2022
Version 7.2	Ergänzung der MMR und Hooks	Marius Reitz	04.01.2022
Version 8.1	Ergänzung der Schnittstellen IRiskeySetCommand, IRiskeySetCommandContext;	Alexander Pohl	10.01.2023
Version 8.2	Ergänzung Export Schnittstellen	René Winkler	22.05.2023
Version 8.2	Ergänzung der Schnittstellen IAbwicklungsVerfahren, INonlifeTariffCalculator; Überarbeitung ISchadensetUpdater;	Alexander Pohl	26.07.2023
Version 8.2	Ergänzung Schnittstellenübersicht	Alexander Pohl	26.07.2023
Version 8.3	Keine Anpassungen gegenüber der Version 8.2	Alexander Pohl	04.10.2023
Version 9.0	Keine Anpassungen gegenüber der Version 8.3	Alexander Pohl	01.12.2023
Version 9.1	Schnittstellen IPlanetUI und IPlanetResultQuery ergänzt	Alexander Pohl	06.03.2024
Version 9.1	Verschiedene Schnittstellen dokumentiert. Allgemeine Überarbeitungen	René Winkler	14.03.2024
Version 9.2	Keine Anpassungen gegenüber der Version 9.1	Alexander Pohl	31.07.2024
Version 10.0	Keine Anpassungen gegenüber der Version 9.2	Alexander Pohl	25.11.2024
Version 10.1	Schnittstelle IRisKeyFilterUserTab ergänzt	Ziyi Zhao	23.01.2025
Version 10.1	Schnittstelle IUserEinzelschadenRunOff ergänzt	René Winkler	14.03.2025
Version 10.1	Schnittstelle IRisKeyFilterCalculationMethod und IRiskeyFilterCalculationMethodContext ergänzt	Alexander Pohl	17.03.2025
Version 10.1	Schnittstelle IAbwDatenRisKey ergänzt	Alexander Pohl	17.03.2025
Version 10.1	Schnittstelle IUserEinzelschadenRunOffContext ergänzt	Alexander Pohl	18.03.2025
Version 11.0	Schnittstelle IRiskeySetCommand, IFilterCommand und IFilterCommandContext ergänzt	Ziyi Zhao	24.11.2025

Inhaltsverzeichnis

1 EINLEITUNG 1
2 ÜBERSICHT 2
3 SCHNITTSTELLEN FÜR RISKEY 4
3.1 Import 4
3.1.1 ISchadensetUpdater 4
3.2 Export 5
3.2.1 IRiskeyQuery 5
3.3 Erweiterte Filterfunktionen 8
3.3.1 IRisKeyFilterUserTab 8
3.3.2 IRisKeyFilterCalculationMethod 8
3.3.3 IRiskeyFilterCalculationMethodContext 9
3.4 Abwicklung 10
3.4.1 IAbwicklungsVerfahren 10
3.4.2 IAbwicklungsverfahrenAusgabe 11
3.4.3 IAbwicklungsverfahrenAusgabe2 11
3.4.4 IAbwDatenRisKey 12
3.5 Reservierung/Sets 12
3.5.1 IRiskeySetCommand 12
3.5.2 IRiskeySetCommandContext 13
3.5.3 IFilterCommand 14
3.5.4 IFilterCommandContext 14
4 SCHNITTSTELLEN FÜR PLA.NET 16
4.1 Allgemeine Steuerung 16
4.1.1 IPlanetUI 16
4.2 Import 18
4.2.1 IBruttoszenarioImporter 18
4.2.2 IRVSzenarioImporter 18
4.2.3 IBruttosznearioUpdater 19
4.2.4 IRVSzenarioUpdater 20
4.3 Export 22
4.3.1 IResultQuery 22
4.3.2 IBatchResultQuery 22
4.3.3 IPlanetResultQuery 23
4.3.4 IResultQueryContext 23
4.4 Managementregeln / Hooks 26
4.4.1 Allgemeines 26
4.4.2 IAdujstVtR 27
4.4.3 IDividendenHook 28

4.5 Tarifklassenprogrammierung 29
4.5.1 INonlifeTariffCalculator 29
4.5.2 INonLifeTariff 29
4.5.3 ITariffClass 30
4.6 Sonstiges 30
4.6.1 IRueckversicherungUI 30
4.6.2 IRegisterRueckversicherung 31
4.6.3 IRegisterSUVCarryProperties 31
4.6.4 ISUVCustomParameter 31
4.6.5 IUserEinzelschadenRunOff 32
4.6.6 IUserEinzelschadenRunOffContext 33

1 EINLEITUNG

In dem vorliegenden Konzept werden die für die Sparte Schaden-/Unfallversicherung (SUV) relevanten Schnittstellen der ALM-Software PLA.NET und des Reservierungstools RisKey erläutert.

Innerhalb des Konzepts wird im Wesentlichen darauf eingegangen, welche Funktionalitäten von der jeweiligen Schnittstelle unterstützt bzw. nicht unterstützt werden.

Außerdem wird kurz auf die für die Sparte SUV relevanten technischen Funktionen der Schnittstellen eingegangen.

2 ÜBERSICHT

Zuerst werden die Schnittstellen übersichtlich in einer Tabelle dargestellt. Anschließend wird auf die einzelnen Schnittstellen eingegangen.

RISKEY:

Anwendungsbereich:	**Schnittstellenbezeichnung:**
Import	ISchadensetUpdater
Export	IRiskeyQuery
Erweiterte Filterfunktionen	IRisKeyFilterUserTab
	IRisKeyFilterCalculationMethod
	IRisKeyFilterCalculationMethodContext
Abwicklung	IAbwicklungsverfahren
	IAbwicklungsverfahrenAusgabe
	IAbwicklungsverfahrenAusgabe2
	IAbwDatenRisKey
Reservierung / Sets	IRisKeySetCommand
	IRisKeySetCommandContext
	IFilterCommand
	IFilterCommandContext

PLA.NET:

Anwendungsbereich:	**Schnittstellenbezeichnung:**
Allgemeine Steuerung	IPlanetUI
Import	IBruttoszenarioImporter
	IRVSzenarioImporter
	IBruttoszenarioUpdater
	IRVSzenarioUpdater
Export	IResultQuery
	IBatchResultQuery
	IPlanetResultQuery
	IResultQueryContext
Managementregeln/Hooks	IAdjustVtRHook
	IDividendenHook
Sonstiges	IRueckversicherungUI
	IRegisterRueckversicherung
	IRegisterSUVCarryProperties
	ISUVCustomParameter
	IUserEinzelschadenRunOff
	IUserEinzelschadenRunOffContext

Übergeordnet:

Anwendungsbereich:	**Schnittstellenbezeichnung:**
Prämienrückstellung	INonlifeTariffCalculator
	INonLifeTariff
	ITariffClass

3 SCHNITTSTELLEN FÜR RISKEY

Im Folgenden werden die Schnittstellen für RisKey genauer erläutert.

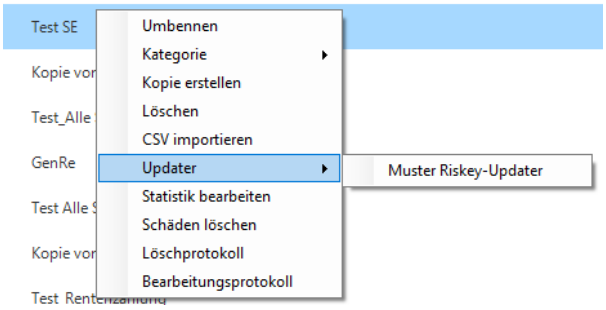
3.1 Import

3.1.1 ISchadensetUpdater

Mittels der Schnittstelle „ISchadensetUpdater“ können Daten automatisiert in ein Schadenset importiert werden. Hierfür muss eine neue Klasse angelegt werden welche folgende Schnittstelle implementiert:

```
Implements ISchadensetUpdater
```

Die Schnittstelle lässt sich im Kontextmenü des Schadensets in RisKey über „Updater“ aufrufen:



Mithilfe der Schnittstelle kann bspw. ein Schadendreieck über eine externe Datei (bspw. eine Excel-Datei) in einen Filter, der zuvor angelegt wurde, geschrieben werden. Innerhalb der Schnittstelle kann man über die Filter ID auf alle Filter des Schadensets zugreifen. Innerhalb des Filters ist es möglich auf alle Abwicklungsdreiecke zuzugreifen (GuV, Prognose, RV-Abgabe, Regulierungskosten, Renten, Bereinigung).

Neben den Abwicklungsdaten können auch Daten für Verteilungsfunktionen oder ähnliches geupdatet werden.

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	***Beschreibung***
`Public Function GetKey() As Guid Implements IUpdater.GetKey`	Liefert eine eindeutige ID (Guid) zurück. Hierfür muss vorab eine Guid definiert worden sein.
`Public Function GetName() As String Implements IUpdater.GetName`	Liefert den individuellen Namen, welcher unter dem Kontextmenüpunkt „Updater“ für den Aufruf der Schnittstelle erscheint.
`Public Function CreateInstance() As IUpdater Implements IUpdater.CreateInstance`	Erzeugt eine Instanz der Importer-Routine.

Funktion / Sub	*Beschreibung*
`Public Function Update(Ctx As ISchadensetUpdaterCtx) As IEnumerable(Of String) Implements ISchadensetUpdater.Update`	Updatefunktion welche beim Aufruf der Schnittstelle aufgerufen wird. Darin können über das Kontext die Informationen abgefragt und importiert werden. Das ISchadensetUpdaterCtx ermöglicht Zugriff auf Riskey Daten eines Schadensets über IRiskeyQuery (siehe IRisKeyQuery)

3.2 Export

3.2.1 IRiskeyQuery

Mittels der Schnittstelle „IRiskeyQuery“ kann auf die Daten eines RisKey Schadensets zugegriffen werden, um diese in eine externe Datei (bspw. eine Excel-Datei) zu exportieren, oder in eine andere Anwendung (bspw. SolvencyII Baum) zu übertragen.

<u>Vorgehensweise</u>
Zunächst muss eine entsprechende SetID ausgewählt werden damit die Daten aus dem „richtigen“ Datensatz von RisKey geladen werden können. Eine komfortable Möglichkeit bietet hier das Interface IUserCalculationUIContext mit folgender Methode:

```
IUserCalculationUIContext.ShowChooseWorksetDlg(PlanetCore.WorkSetNames.RKData)
```

Beim Ausführen dieser Methode öffnet sich ein Auswahldialog bestehender RisKeysets. Der Rückgabetyp dieser Funktion ist ein Integer (eben diese ID) mit welcher die Daten im Folgenden geladen werden können.

```
Dim RKQuerySource = IUserCalculationUIContext.CreateRiskeyQueryAsync(ID)
```

```
Dim RKQuery As PlanetCore.IRiskeyQuery = Nothing
RKQuery = Await RKQuerySource
```

Hinweis:
Mittels CreateRisKeyQueryAsync(ID) wird eine Verbindung zum RisKeyset geschaffen. Mittels der asynchronen Methode (Await RKQuerySource) werden grundlegende Daten des Schadensets geladen. Nicht aber die Daten aller Filter und Sets.

Die Daten der Sets werden mit folgender Methode geladen:

```
Await RKQuery.LoadFilterSetAsync(RisKeySetKey)
```

Dabei ist der RisKeySetKey ein Key (bzw. eine Klasse deren Attribute einen Key bilden) dessen Ausprägung genau einen Knoten des Reservierungsbaums in RisKey beschreibt. Wie man der Methode anerkennen kann, muss jedes Filterset mittels dieser asynchronen Methode und einem angepassten RisKeySetKey geladen werden.

Für die weitere Datenabfrage helfen folgende Funktionen:

Funktion / Sub	*Beschreibung*
`Public Function GetSpartenDefMgr() As SpartenDefMgr Implements IRiskeyQuery.GetSpartenDefMgr`	Liefert den Spartendefinitionsmanager zur aktuell verwendeten Spartendefinition.
`Public Function GetFilter(FilterID As Integer) As Filter Implements IRiskeyQuery.GetFilter`	Liefert den Filter mit der entsprechenden ID.
`Public Async Function GetFilterAsync(FilterID As Integer) As Task(Of Filter) Implements IRiskeyQuery.GetFilterAsync`	Liefert den Filter mit der entsprechenden ID.
`Public Async Function LoadFilterSetAsync(filterSet As RisKeySet) As Task Implements IRiskeyQuery.LoadFilterSetAsync`	Lädt die Daten des übergebenen RisKeySets.
`Public Function GetExistingYears() As Integer() Implements IRiskeyQuery.GetExistingYears`	Liefert die angelegten Jahre im Solvency II Reservierungsbaum.
`Public Function HasMeldung(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen) As Boolean Implements IRiskeyQuery.HasMeldung`	Gibt zurück, ob für das übergebene Jahr die übergebene Meldung existiert.
`Public Function HasMeldung(Jahr As Integer, MeldungVon As RisKeySet.SII_Meldungen, MeldungBis As RisKeySet.SII_Meldungen) As Boolean Implements IRiskeyQuery.HasMeldung`	Gibt zurück, ob für das übergebene Jahr die übergebenen Meldungen existiert. Gibt „True“ zurück, wenn alle Meldungen existieren, sonst „False“.
`Public Function GetMeldungen(Jahr As Integer) As RisKeySet.SII_Meldungen() Implements IRiskeyQuery.GetMeldungen`	Gibt alle Meldungen für das übergebene Jahr zurück.
`Public Function GetLoBs(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen) As SolvencyIILoBs() Implements IRiskeyQuery.GetLoBs`	Gibt die unterhalb der übergebenen Meldung für das übergebene Jahr angelegten LoBs zurück.
`Public Function HasFilterSet(Key As RiskeySetKey) As Boolean Implements IRiskeyQuery.HasFilterSet`	Gibt zurück, ob ein RiskeySet zum entsprechenden RiskeySetKey existiert.
`Public Function GetHomogeneRisikogruppen(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen, LoB As SolvencyIILoBs) As List(Of HomogeneRisikogruppe) Implements IRiskeyQuery.GetHomogeneRisikogruppen`	Gibt eine Liste der angelegten homogenen Risikogruppen in Abhängigkeit vom Jahr, der Meldung und der LoB zurück.
`Public Function GetUnterstuetzungsarten(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen, LoB As SolvencyIILoBs) As KompositSolvencyHelper.RisKeySIIResUnterstuetzung() Implements IRiskeyQuery.GetUnterstuetzungsarten`	Gibt die Unterstützungsarten in Abhängigkeit von Jahr, Meldung und LoB zurück.
`Public Function GetUnterstuetzungsarten(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen, LoB As SolvencyIILoBs, HRG As HomogeneRisikogruppe) As KompositSolvencyHelper.RisKeySIIResUnterstuetzung() Implements IRiskeyQuery.GetUnterstuetzungsarten`	Gibt die Unterstützungsarten in Abhängigkeit von Jahr, Meldung, LoB und HRG zurück.

Funktion / Sub	*Beschreibung*
`Public Function GetBruttoNetto(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen, LoB As SolvencyIILoBs, UA As KompositSolvencyHelper.RisKeySIIResUnterstuetzung) As KompositSolvencyHelper.RiskeyBruttoNetto() Implements IRiskeyQuery.GetBruttoNetto`	Gibt KompositSolvencyHelper.RiskeyBruttoNetto() in Abhängigkeit von Jahr, Meldung, LoB und Unterstützungsart zurück.
`Public Function GetBruttoNetto(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen, LoB As SolvencyIILoBs, HRG As HomogeneRisikogruppe, UA As KompositSolvencyHelper.RisKeySIIResUnterstuetzung) As KompositSolvencyHelper.RiskeyBruttoNetto() Implements IRiskeyQuery.GetBruttoNetto`	Gibt KompositSolvencyHelper.RiskeyBruttoNetto() in Abhängigkeit von Jahr, Meldung, LoB, HRG und Unterstützungsart zurück.
`Public Function GetFilterliste(Key As RiskeySetKey) As List(Of Filter) Implements IRiskeyQuery.GetFilterliste`	Gibt eine Liste der Filter in Abhängigkeit vom RiskeySetKey zurück.
`Public Function GetFilterliste(Key As RiskeySetKey, UnterstArt As KompositSolvencyHelper.RisKeySIIResUnterstuetzung?, BruttoNetto As KompositSolvencyHelper.RiskeyBruttoNetto?) As List(Of Filter) Implements IRiskeyQuery.GetFilterliste`	Gibt eine Liste der Filter in Abhängigkeit von RiskeySetKey, Unterstützungsart und KompositSolvencyHelper.RiskeyBruttoNetto zurück.
`Public Function GetFilterliste(Jahr As Integer, Meldung As RisKeySet.SII_Meldungen?, LoB As SolvencyIILoBs?, HRG As HomogeneRisikogruppe, UnterstArt As KompositSolvencyHelper.RisKeySIIResUnterstuetzung?, BruttoNetto As KompositSolvencyHelper.RiskeyBruttoNetto?) As List(Of Filter) Implements IRiskeyQuery.GetFilterliste`	Gibt eine Liste der Filter in Abhängigkeit von Jahr, Meldung, LoB, HRG, Unterstützungsart und KompositSolvencyHelper.RiskeyBruttoNetto zurück.
`Public Function GetRisikomarge(Key As RiskeySetKey) As RiskeySetRisikomarge Implements IRiskeyQuery.GetRisikomarge`	Gibt die Risikomarge in Abhängigkeit vom RiskeySetKey zurück
`Public Function GetRisKeySetbyKey(Key As RiskeySetKey) As RisKeySet Implements IRiskeyQuery.GetRisKeySetbyKey`	Gibt das RiskeySet zum übergebenen RiskeySetKey zurück.
`Public Function GetPremprovision(Key As RiskeySetKey, BruttoNetto As KompositSolvencyHelper.RiskeyBruttoNetto) As RisKeySetPrRstVereinfacht Implements IRiskeyQuery.GetPremprovision`	Gibt die Prämienrückstellungskomponenten nach der Vereinfachungsmethode in Abhängigkeit von RiskeySetKey und KompositSolvencyHelper.RiskeyBruttoNetto zurück.
`Public Function GetFreieFilter() As Filter()Implements IRiskeyQuery.GetFreierFilter`	Gibt den entsprechenden Filter, die unter der Reiter „Filter“ sind, zurück.
`Public Function GetOrAddMarktabwicklung(LoB As KompositSolvencyHelper.MarktabwicklungLoBs) As Marktabwicklung Implements IRiskeyQuery.GetOrAddMarktabwicklung`	Gibt die entsprechende Marktabwicklung in Abhängigkeit von LoB
`Public Function GetOrAddVolumen(SparteID As Integer, Jahr As Integer) As Volumen Implements IRiskeyQuery.GetOrAddVolumen`	Gibt die entsprechende Volumenmaße in Abhängigkeit von SparteID und Jahr zurück.
`Public Function GetSparten() As IEnumerable(Of Sparte) Implements IRiskeyQuery.GetSparten`	Gibt eine Liste der angelegten Sparten zurück.

Funktion / Sub	*Beschreibung*
`Public Function GetPauschaldreiecke() As IEnumerable(Of Pauschaldreieck) Implements IRiskeyQuery.GetPauschaldreiecke`	Gibt eine Liste der angelegten Pauschaldreiecke zurück.

3.3 Erweiterte Filterfunktionen

3.3.1 IRisKeyFilterUserTab

Mittels der Schnittstelle „IRisKeyFilterUserTab“ können individuelle, benutzerdefinierte Registerkarten innerhalb der Abwicklungsanalyse erstellt werden. Diese können benutzerdefinierte Ergebnistypen enthalten, für welche eine Ausgabemaske zu definieren ist.

Folgende Eigenschaften und Funktionen müssen implementiert werden:

Funktion / Sub	*Beschreibung*
`Function GetName() As String`	Gibt den Namen der Registerkarte zurück
`Function GetKey() As Guid`	Gibt den technischen Schlüssel des Ergebnistyps zurück. Hierbei muss es sich um eine eindeutige ID (Guid) handeln.
`Function CreateInstance() As IRiskeyFilterUserTab`	Technische benötigte Funktion. Gibt ein konkretes Object aus dieser Schnittstelle zurück.
`ReadOnly Property FrozenRows As Integer`	Anzahl der fixierten Zeilen der Ausgabemaske des benutzerdefinierten Ergebnistyps
`ReadOnly Property FrozenColumns As Integer`	Anzahl der fixierten Spalten der Ausgabemaske des benutzerdefinierten Ergebnistyps
`Sub PrepareRangeGrid(rgc As IRangeGridControl, filter As Filter)`	Definition der Ausgabemaske des benutzerdefinierten Ergebnistyps

3.3.2 IRisKeyFilterCalculationMethod

Die Schnittstelle „IRisKeyFilterCalculationMethod“ bietet die Funktionalität für benutzerindividuelle Berechnungsmethoden auf Basis der im Filter selektierten Einzelschadendaten. Es kann zwischen den *CalculationMethodTypes* „PauschalDreieck“ und „ULAE“ gewählt werden. Der ULAE-Typ bietet die Möglichkeit den Zahlungsstrom für indirekte Regulierungskosten nach einer individuellen Berechnungslogik zu ermitteln und die Ergebnisse im Bereich „Abwicklung Cashflows“ der Abwicklungsanalyse auszuweisen.

Die Schnittstelle stellt folgende Funktionalitäten zur Verfügung:

Funktion / Sub	*Beschreibung*
`Public Function GetName() As String Implements IRisKeyFilterCalculationMethod.GetName`	Gibt den Namen zurück, welcher der Berechnungsmethode zugewiesen wurde.
`Public Function GetKey() As Guid Implements IRisKeyFilterCalculationMethod.GetKey`	Gibt den technischen Schlüssel der Berechnungs-Routine zurück. Hierbei muss es sich um eine eindeutige ID (Guid) handeln.
`Public Function CreateInstance() As IRisKeyFilterCalculationMethod Implements IRisKeyFilterCalculationMethod.CreateInstance`	Erzeugt eine neue Instanz der Berechnungs-Routine. Dabei handelt es sich in der Regel um eine Instanz der Klasse, welche die Schnittstelle implementiert.
`Public Function GetMethodType() As CalculationMethodTypes Implements IRisKeyFilterCalculationMethod. GetMethodType`	Funktion, welche den Typ der Berechnungsmethode zurückgibt.
`Public Async Function ExecuteAsync(context As IRiskeyFilterCalculationMethodContext, filter As Filter) As Task Implements IRisKeyFilterCalculationMethod.ExecuteAsync`	Steuerung und Umsetzung des Berechnungsablaufs. Innerhalb dieser Funktion erfolgt die konkrete Umsetzung der benutzerdefinierten Programmierung.

3.3.3 IRiskeyFilterCalculationMethodContext

Mittels der Schnittstelle „IRiskeyFilterCalculationMethodContext“, welche als Funktionsparameter der Funktion *IRisKeyFilterCalculation-Method.ExecuteAsync()* übergeben wird, wird der Zugriff und das Laden der notwendigen Modellparamter für die Berechnungsmethode ermöglicht.

Die Schnittstelle stellt folgende Funktionalitäten zur Verfügung:

Funktion / Sub	*Beschreibung*
`Public Function EnsureClaimsData (Optional ForceReload As Boolean = False) As Task`	Lädt die Schadendaten des Filter-Modells, sofern dies noch nicht erfolgt ist. Optional kann ein erneutes Laden der Daten auch erzwungen werden.
`Public Function EnsureAuspraegungen() As Task`	Lädt die Ausprägungen der Schadendaten über das Filter-Modell (Sparte, Komponente, Gefahr, …)
`Public Function EnsureSchadenEreignisse() As Task`	Lädt die Schadenereignisse über das Filter-Modell.
`Public Function GetSchadenSet() As Schadenset`	Lädt das Schadenset über das Filter-Modell.
`Public Function GetUdtMgr() As UserDatatypeManager`	Gibt den UserDatatypeManager für den Zugriff auf die Benutzerdaten zurück.

3.4 Abwicklung

3.4.1 IAbwicklungsVerfahren

Mittels der Schnittstelle IAbwicklungsVerfahren können eigene Abwicklungsverfahren geschrieben werden. Dies kann aufgrund unternehmensindividueller Anforderungen wie bspw. der Notwendigkeit von dreidimensionalen Verfahren der Fall sein.

Abwicklungsverfahren werden grundsätzlich mit der Schnittstelle IAbwicklungsVerfahren implementiert. Hierfür muss zuvor der Namespace PlanetAbwicklung eingebunden werden.

```
Implements IAbwicklungsVerfahren
```

Folgende Eigenschaften und Funktionen müssen implementiert werden

Funktion / Sub	***Beschreibung***
`ReadOnly Displayname As String`	Gibt den Namen des Abwicklungsverfahrens zurück
`Sub Execute(Context As IAbwicklungContext, Daten As IAbwDaten, StartFJ As Integer, LetztesAJ As Integer, Art As Verfahrensart)`	Sub welche für die Berechnung des Verfahrens herangezogen wird. Mittels der Schnittstelle IAbwDaten erfolgt dabei der Datentransfer zur Schnittstelle. Die Parameter StartFJ und LetztesAJ sind nur für Aufrufe aus PLA.NET notwendig.
`Sub LoadParameter(data() As Byte)`	Sub welche die (eigenen) Parameter des Abwicklungsverfahrens aus der Datenbank lädt.
`Sub LoadParameter(reader As XmlReader)`	Sub welche die (eigenen) Parameter des Abwicklungsverfahrens aus einem XML-File lädt.
`Function SaveParameter() As Byte()`	Function welche die (eigenen) Parameter des Abwicklungsverfahrens für das Speichern in einer Datenbank vorbereitet.
`Sub SaveParameter(writer As XmlWriter)`	Sub welche die (eigenen) Parameter des Abwicklungsverfahrens für das Speichern in einem XML-File vorbereitet.
`Sub SetAbwicklungsfaktoren(Schadenzahlung As AbwicklungsViereck, Abwicklungsfaktoren() As Double)`	Derzeit nicht verwendet
`Function GetKey() As Guid`	Eindeutiger Schlüssel / ID der Schnittstelle vom Typ Guid
`Function GetName() As String`	Function liefert an der Oberfläche den vorgegebenen Namen des Abwicklungsverfahrens zurück. Dieser wird in der Auswahl der Abwicklungsverfahren angezeigt.
`Function CreateInstance() As IAbwicklungsverfahren`	Technisch benötigte Function, welche ein konkretes Objekt des Abwicklungsverfahrens zurückgibt.
`Function GetVerfahrensart() As Verfahrensart`	Mittels dieser Function kann man die Verfahrensart festlegen. Hier kann man bspw. festlegen, dass das

Funktion / Sub	*Beschreibung*
	Abwicklungsverfahren auf Schadenzahlungen, Reserven oder Aufwände angewendet werden soll. Entsprechend der Definition erscheint dann auch nur der Name des Abwicklungsverfahrens im jeweiligen Reiter der Abwicklungsmaske.

3.4.2 IAbwicklungsverfahrenAusgabe

Diese Schnittstelle steuert die Benutzerprogrammierung für die Ausgabe von Abwicklungsverfahren mittels eines RangeGridControl. Unter „…“ neben „Zusatzausgaben“ im Reiter Abwicklungsverfahren aufgerufen werden, nachdem das Abwicklungsverfahren ausgewählt wurde.

Die Schnittstelle stellt folgende Funktionalitäten zu Verfügung:

Funktion / Sub	*Beschreibung*
`ReadOnly Property FrozenRows As Integer`	Gibt die Anzahl an fixierten Zeilen der Ausgabemaske des Abwicklungsverfahres zurück
`ReadOnly Property FrozenColumns As Integer`	Gibt die Anzahl an fixierten Spalten der Ausgabemaske des Abwicklungsverfahres zurück
`Sub PrepareRangeGrid(rgc As IRangeGridControl)`	In diesem Sub wird die Ausgabemaske des Abwicklungsverfahrens definiert. Als Übergabeparameter ist ein RangeGrid nötig.

3.4.3 IAbwicklungsverfahrenAusgabe2

Die Schnittstelle „IAbwicklungsverfahrenAusgabe2“ macht es möglich, eigene Ausgabedialoge für Abwicklungsverfahren zu programmieren. Diese können unter „…“ neben „Zusatzausgaben“ im Reiter Abwicklungsverfahren aufgerufen werden, nachdem das Abwicklungsverfahren ausgewählt wurde.

Die Schnittstelle stellt zwei Funktionalitäten zur Verfügung:

Funktion / Sub	*Beschreibung*
`Public Sub ShowAusgabe(UserFormManager As IUserFormManager, AbwVerfahren As IAbwicklungsverfahren, Daten As IAbwDaten) Implements IAbwicklungsverfahrenAusgabe2.ShowAusgabe`	Hier kann die Windows Form aufgerufen werden, die angezeigt werden soll.
`Public Function GetKey() As Guid Implements IAbwicklungsverfahrenAusgabe2.GetKey`	Hier wird die Ausgabe mit dem gewünschten Abwicklungsverfahren verknüpft, indem die Guid des Abwicklungsverfahrens zurückgegeben wird.

3.4.4 IAbwDatenRisKey

Mittels der Schnittstelle IAbwDatenRisKey können Abwicklungsmethoden nun auch auf Daten aus anderen Filtern zugreifen. Dies ermöglicht die Aufbereitung einer Datenbasis oder die Parametrisierung eines Abwicklungsverfahrens aus vorangegangenen, abweichenden Filtereinstellungen und der daraus abgeleiteten Eingangsgrößen.

Die Schnittstelle bietet folgenden Funktionsumfang:

Funktion / Sub	***Beschreibung***
`Public Function GetSchadensetView() As ISchadensetView`	Gibt das aktuell aktive SchadensetView zurück.
`Public Function GetSchadenSet() As Schadenset`	Gibt das Schadenset aus dem SchadensetView zurück.
`Public Function GetZinsstruktur() As Zinsstruktur`	Gibt die Zinsstruktur aus dem SchadensetView zurück.
`Public Function GetFilter() As Filter`	Gibt den aktuell ausgewählten Filter zurück.
`Public Function GetFilter(filterId As Integer) As Filter`	Gibt den Filter zur übergebenen Filter-ID zurück.
`Public Function GetFilterSiblings() As Filter()`	Gibt eine Liste von Filtern zurück mit der gleichen Primärzuordnung zum Reservierungsbaumknoten wie der aufrufende Filter.
`Public Function GetFilterSecondarySiblings() As Filter()`	Gibt eine Liste von Filtern zurück mit der gleichen Sekundärzuordnung zum Setsbaumknoten wie der aufrufende Filter.

3.5 Reservierung / Sets

3.5.1 IRiskeySetCommand

Die Schnittstelle „IRiskeySetCommand“ ermöglicht weitere Kontextmenüeinträge auf allen Ebenen des Reservierungsbaumes oder der Sets hinzuzufügen. Darunter können anschließend benutzerdefinierte Berechnungs-Routinen im Bereich Reservierung / Sets erfolgen, für welche ebenso benutzerdefinierte Ein- und Ausgabedialoge programmiert werden können.

Der Zugriff auf die für die benutzerdefinierten Berechnungen gegebenenfalls benötigten Daten aus dem Reservierungsbaum / Sets erfolgt über die Verwendung der Schnittstelle „IRiskeyQuery“ (vgl. hierzu Abschnitt 3.2.1).

Die Schnittstelle stellt folgende Funktionalitäten zur Verfügung:

Funktion / Sub	*Beschreibung*
`Public Function GetName() As String Implements IRiskeySetCommand.GetName`	Gibt den Namen zurück, welcher dem Kontextmenüeintrag zugewiesen werden soll.
`Public Function GetKey() As Guid Implements IRiskeySetCommand.GetKey`	Gibt den technischen Schlüssel der Berechnungs-Routine zurück. Hierbei muss es sich um eine eindeutige ID (Guid) handeln.
`Public Function CreateInstance() As IRiskeySetCommand Implements IRiskeySetCommand.CreateInstance`	Erzeugt eine neue Instanz der Berechnungs-Routine. Dabei handelt es sich in der Regel um eine Instanz der Klasse, welche die Schnittstelle implementiert.
`Public Function ShowInContextMenu() As Boolean Implements IRiskeySetCommand.ShowInContextMenu`	Macht den Kontextmenüeintrag an der Oberfläche im Reservierungsbaum/Setsbaum auf der/den gewünschten Ebene/-n sichtbar.
`Public Async Function ExecuteAsync(Context As IRiskeySetCommandContext) As Task Implements IRiskeySetCommand.ExecuteAsync`	Steuerung und Umsetzung der Programmroutine. Innerhalb dieser Funktion erfolgt die konkrete Umsetzung der benutzerdefinierten Programmierung.

Anwendungsbeispiele:

- Veränderungsanalyse
- Präminerückstellung

3.5.2 IRiskeySetCommandContext

Die Schnittstelle „IRiskeySetCommandContext“ steuert das Öffnen und Schließen von Dialogen, den Datenzugriff, sowie das Laden und Speichern von Daten und Ergebnissen innerhalb einer benutzerdefinierten Berechnungs-Routine (vgl. hierzu Abschnitt 3.5.1). Der Zugriff auf die Schnittstelle erfolgt über den Parameter der Funktion *IRiskeySetCommand. ExecuteAsync()*.

Die Schnittstelle stellt folgende Funktionalitäten zur Verfügung:

Funktion / Sub	*Beschreibung*
`Sub SetChanged()`	Setzt den Status des Speicher-Icons auf „Aktiv“. Anmerkung: Manche Anpassungen werden nicht allgemein als Änderungen wahrgenommen und würden beim Schließen des Dialoges oder beim Beenden der Berechnungs-Routine verloren gehen. Mittels dieser Sub können aktiv Anpassungen signalisiert werden.
`Sub ShowMDIChildForm(form As Form)`	Öffnet einen Dialog vom Typ Windows.Forms.Form.
`Sub CloseMDIChildForm(form As Form)`	Schließt einen Dialog vom Typ Windows.Forms.Form.
`Function GetCurrentRiskeySet() As RisKeySet`	Gibt das aktuelle RisKey-Set zurück.
`Function GetRiskeyQueryAsync() As Task(Of IRiskeyQuery)`	Gibt einen Task vom Typ IRiskeyQuery zurück.

Funktion / Sub	*Beschreibung*
`Function SaveCurrentRiskeySet() As Task`	Speichert die Änderungen im aktuellen RisKey-Set.
`Function LoadAllFiltersAsync (requireExclusive As Boolean) As Task`	Lädt alle dem RisKey-Set zugeordneten Filter.

3.5.3 IFilterCommand

Die Schnittstelle „IFilterCommand“ ermöglicht erweiterte Kontextmenüeinträge auf Filter-Ebene hinzuzufügen. Darunter können anschließend benutzerdefinierte Berechnungs- und Ablaufroutinen im Bereich Reservierung / Sets erfolgen, für welche ebenso benutzerdefinierte Ein- und Ausgabemasken programmiert werden können.

`Public Function GetName() As String Implements IFilterCommand.GetName`	Gibt den Namen zurück, welcher dem Kontextmenüeintrag zugewiesen werden soll.
`Public Function GetKey() As Guid Implements IFilterCommand.GetKey`	Gibt den technischen Schlüssel der Berechnungs-Routine zurück. Hierbei muss es sich um eine eindeutige ID (Guid) handeln.
`Public Function CreateInstance() As IRiskeySetCommand Implements IFilterCommand.CreateInstance`	Erzeugt eine neue Instanz der Berechnungs-Routine. Dabei handelt es sich in der Regel um eine Instanz der Klasse, welche die Schnittstelle implementiert.
`Public Function ShowInContextMenu() As Boolean Implements IFilterCommand.ShowInContextMenu`	Macht den Kontextmenüeintrag an der Oberfläche im Bereich Reservierung / Sets auf den gewünschten Filtern (gemäß dem zugewiesenen RiskeySet) sichtbar.
`Public Async Function ExecuteAsync(Context As IFilterCommandContext) As Task Implements IFilterCommand.ExecuteAsync`	Steuerung und Umsetzung des Berechnungsablaufs. Innerhalb dieser Funktion erfolgt die konkrete Umsetzung der benutzerdefinierten Programmierung.

3.5.4 IFilterCommandContext

Die Schnittstelle „IFilterCommandContext“ steuert das Öffnen und Schließen von Dialogen, den Datenzugriff, sowie das Laden und Speichern von Daten und Ergebnissen innerhalb einer benutzerdefinierten Berechnungs-Routine (vgl. hierzu Abschnitt 3.5.1). Der Zugriff auf die Schnittstelle erfolgt über den Parameter der Funktion *IFilterCommand. ExecuteAsync()*.

Die Schnittstelle stellt folgende Funktionalitäten zur Verfügung:

Funktion / Sub	*Beschreibung*
`Function GetCurrentFilter() As Filter`	Gibt die aktuellen Filter zurück, welcher im Bereich Reservierung / Sets ausgewählt ist.
`Function GetRiskeyQueryAsync() As Task(Of IRiskeyQuery)`	Gibt einen Task vom Typ IRiskeyQuery zurück
`Sub SetChanged()`	Markiert das aktuelle Riskey-Set als verändert und gibt das Speicher-Icon frei.

Funktion / Sub	*Beschreibung*
`Function SaveCurrentFilter() As Task`	Speichert den aktuellen Filter.
`Sub ShowMDIChildForm(form As Form)`	Öffnet einen Dialog des Typen Windows.Forms.Form.
`Sub CloseMDIChildForm(form As Form)`	Schließt einen Dialog des Typen Windows.Forms.Form.

4 SCHNITTSTELLEN FÜR PLA.NET

Im Folgenden werden die Schnittstellen für PLA.NET erläutert.

4.1 Allgemeine Steuerung

4.1.1 IPlanetUI

Mit dieser Schnittstelle können allgemeine Informationen abgerufen und grundlegende PLA.NET-Einstellungen vorgenommen werden.

```
Implements IPlanetUI
```

Folgende Funktionen und Methoden müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Function GetRevision() As Integer`	Gibt die Revisionsnummer von PLA.NET zurück.
`Function GetBuilddatum() As String`	Gibt das Erzeugungsdatum von PLA.NET zurück.
`Function GetDatabaseName() As String`	Gibt den Namen der verbundenen PLA.NET-Datenbank zurück.
`Function GetCurrentUser() As String`	Gibt den Namen des aktuellen PLA.NET-Benutzers zurück.
`Function GetCurrentUserAsync() As Task(Of String)`	Gibt den Namen des aktuellen PLA.NET-Benutzers zurück (asynchron).
`Function GetCurrentUserLogin() As String`	Gibt den Anmeldenamen des aktuellen PLA.NET-Benutzers zurück.
`Function GetCurrentUserLoginAsync() As Task(Of String)`	Gibt den Anmeldenamen des aktuellen PLA.NET-Benutzers zurück (asynchron).
`Function GetMainformHandle() As IWin32Window`	Gibt das Fensterhandle des Hauptfensters von PLA.NET zurück.
`Function ShowChooseBatchDlg() As Integer`	Zeigt einen Dialog zur Auswahl eines existierenden Batchlaufs an.
`Function ShowChooseBatchDlgAsync() As Task(Of Integer)`	Zeigt einen Dialog zur Auswahl eines existierenden Batchlaufs an (asynchron).
`Function ShowChooseProfilDlg() As Integer`	Zeigt einen Dialog zur Auswahl eines existierenden Profils an.
`Function ShowChooseProfilDlgAsync() As Task(Of Integer)`	Zeigt einen Dialog zur Auswahl eines existierenden Profils an (asynchron).
`Function ShowChooseWorksetDlg(SetName As String) As Integer`	Zeigt einen Dialog zur Auswahl eines existierenden Datensets mit angegebenen Datensettyp an.

Funktion / Sub	*Beschreibung*
`Function ShowChooseWorksetDlgAsync(SetName As String) As Task(Of Integer)`	Zeigt einen Dialog zur Auswahl eines existierenden Datensets mit angegebenen Datensettyp an (asynchron).
`Function ShowChooseResultDlg() As Integer`	Zeigt einen Dialog zur Auswahl eines existierenden Ergebnisses an.
`Function ShowChooseResultDlgAsync() As Task(Of Integer)`	Zeigt einen Dialog zur Auswahl eines existierenden Ergebnisses an (asynchron).
`Function OpenWorkDb() As IAlmWorkDb`	Öffnet eine neue Datenbankverbindung zur Arbeitsdatenbank.
`Function OpenWorkDbAsync() As Task(Of IAlmWorkDb)`	Öffnet eine neue Datenbankverbindung zur Arbeitsdatenbank (asynchron).
`Function CreateRiskeyQueryAsync(SetID As Integer) As Task(Of IRiskeyQuery)`	Erzeugt eine Riskey-Datenabfrage zu angegebenen Schadendaten-Set (asynchron).
`Function CreateRiskeyQueryForProfi-lAsync(ProfilID As Integer) As Task(Of IRiskeyQuery)`	Erzeugt eine Riskey-Datenabfrage zu angegebenem Profil (asynchron).
`Sub ShowInfoScreen(MessageText As String, Optional ActionText As String = Nothing)`	Zeigt den Warte-Dialog an.
`Sub CloseInfoScreen()`	Schließt den Warte-Dialog.
`Sub ShowWaitCursor()`	Ändert den Mauscursor auf den Wartecursor.
`Sub HideWaitCursor()`	Ändert den Mauscursor auf den Standardcursor.
`Sub ExecuteOnUIThread(UIAction As Action, Synchron As Boolean)`	Führt die übergebene Funktion im UI-Thread aus.
`Function ExecuteOnUIThread(Of T)(UIFunc As Func(Of T)) As T`	Führt die übergebene Funktion im UI-Thread aus und gibt anschließend das Ergebnis zurück.
`Function QueueUserCalculationExecution(uce As IUserCalculationExecution, Stngs As Us-erCalculationExecutionSettings) As Integer`	Reiht eine neue benutzerdefinierte Berechnungsfunktion in die Berechnungswarteschlange ein.
`Sub CancelUserCalculationExecution(calcu-lationId As Integer, localCalculation As Boolean)`	Bricht eine benutzerdefinierte Berechnungsfunktion ab.
`Function ShowNewWorksetDlg(SetName As String, InitialName As String) As ISetDaten`	Zeigt einen Dialog zum Hinzufügen eines neuen Sets an.
`Function RefreshDatasAsync(showWaitForm As Boolean) As Task`	Aktualisiert den PLA.NET-Datenbaum (asynchron).
`Function RefreshResultsAsync(showWaitForm As Boolean) As Task`	Aktualisiert den PLA.NET-Ergebnisbaum (asynchron).
`Function RefreshBatchAsync(showWaitForm As Boolean) As Task`	Aktualisiert den PLA.NET-Batchbaum (asynchron).
`Function GetTariffClassMgr() As ITa-riffClassMgr`	Gibt eine Referenz auf den Tarifklassen-Manager zurück.
`Sub SendHinweis(Hinweis As PlanetMessage)`	Sendet eine Meldung an die Oberfläche.

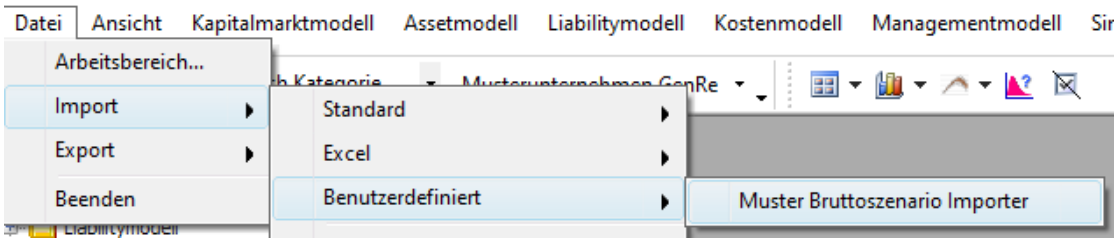
4.2 Import

4.2.1 IBruttoszenarioImporter

Um ein neues Bruttoszenario in PLA.NT anlegen und befüllen zu können, ist in einer neuen Klasse die Schnittstelle „IBruttoszenarioImporter“ zu implementieren:

```
Implements IBruttoszenarioImporter
```

Die Schnittstelle wird über den Menüpunkt „Datei“ → „Import“ → „Benutzerdefiniert“ aufgerufen:



Mithilfe der Schnittstelle kann ein neues Bruttoszenario erzeugt und über eine externe Datei (bspw. eine Excel-Datei) befüllt werden. Ein bestehendes Bruttoszenario kann nicht befüllt bzw. überschrieben werden.

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

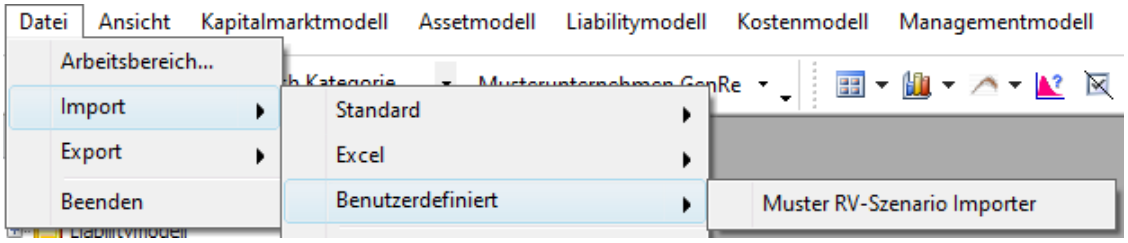
Funktion / Sub	***Beschreibung***
`Public Function GetKey() As Guid Implements IImporter.GetKey`	Liefert eine eindeutige ID (Guid) zurück. Hierfür muss vorab eine Guid definiert worden sein.
`Public Function GetName() As String Implements IImporter.GetName`	Liefert den individuellen Namen, welcher unter dem Menüpunkt „Datei“ → „Import“ → „Benutzerdefiniert“ für den Aufruf der Schnittstelle erscheint.
`Public Function GetFilter() As String Implements IImporter.GetFilter`	Liefert die Dateitypen, welche für die zur Befüllung des Bruttoszenarios zu importierende Datei unterstützt werden.
`Public Function CreateInstance() As IUpdater Implements IImporter.CreateInstance`	Erzeugt eine Instanz der Importer-Routine.

4.2.2 IRVSzenarioImporter

Um ein neues Rückversicherung(RV)- Szenario in PLA.NT anlegen und befüllen zu können, ist die Schnittstelle „IRVSzenarioImporter“ zu implementieren:

```
Implements IRVSzenarioImporter
```

Die Schnittstelle wird über den Menüpunkt „Datei“ → „Import“ → „Benutzerdefiniert“ aufgerufen:



Mithilfe der Schnittstelle kann ein neues RV-Szenario erzeugt und über eine externe Datei (bspw. eine Excel-Datei) befüllt werden. Ein bestehendes RV-Szenario kann nicht befüllt bzw. überschrieben werden.

Hinweis: Da die zugrundeliegende Programmierung der RV-Szenario Importer-Routine aufwendig und komplex ist, ist es oft effizienter die Befüllung des RV-Szenarios manuell vorzunehmen.

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

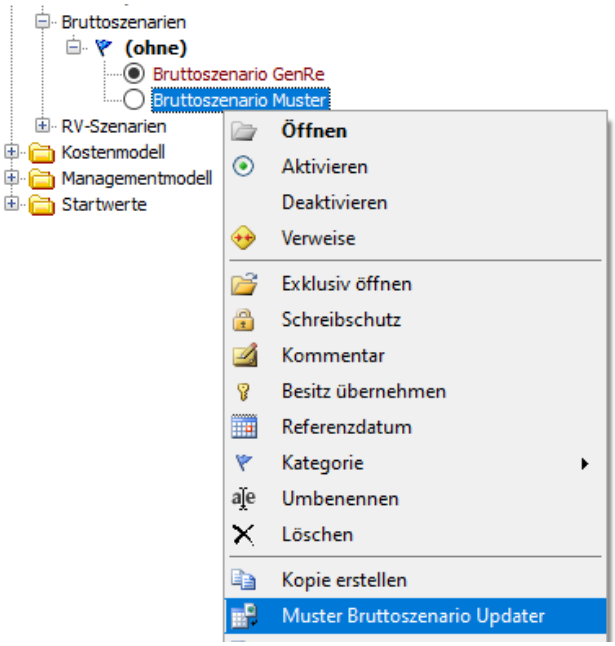
Funktion / Sub	***Beschreibung***
`Public Function GetKey() As Guid Implements IImporter.GetKey`	Liefert eine eindeutige ID (Guid) zurück. Hierfür muss vorab eine Guid definiert worden sein.
`Public Function GetName() As String Implements IImporter.GetName`	Liefert den individuellen Namen, welcher unter dem Menüpunkt „Datei“ → „Import“ → „Benutzerdefiniert“ für den Aufruf der Schnittstelle erscheint.
`Public Function GetFilter() As String Implements IImporter.GetFilter`	Liefert die Dateitypen, welche für die zur Befüllung des RV-Szenarios zu importierende Datei unterstützt werden.
`Public Function CreateInstance() As IUpdater Implements IImporter.CreateInstance`	Erzeugt eine Instanz der Importer-Routine.

4.2.3 IBruttosznearioUpdater

Für die automatisierte Befüllung eines bestehenden Bruttoszenarios in PLA.NT ist die Schnittstelle „IBruttoszenarioUpdater“ zu implementieren:

```
Implements IBruttoszenarioUpdater
```

Die Schnittstelle wird über das Kontextmenü eines bestehenden Bruttoszenarios aufgerufen:



Es kann jede Position der einzelnen Reiter des Bruttoszenarios über eine externe Datei (bspw. eine Excel-Datei) befüllt und ggf. überschrieben werden. Das Erzeugen eines neuen Bruttoszenarios ist nicht möglich.

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

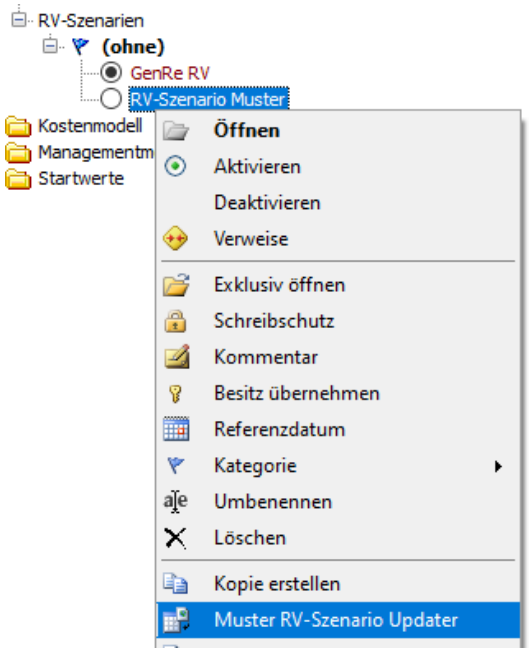
Funktion / Sub	***Beschreibung***
`Public Function GetKey() As Guid Implements IUpdater.GetKey`	Liefert eine eindeutige ID (Guid) zurück. Hierfür muss vorab eine Guid definiert worden sein.
`Public Function GetName() As String Implements IUpdater.GetName`	Liefert den individuellen Namen zum Aufruf der Updater-Routine, welcher unter dem Kontextmenü-Eintrags eines Bruttoszenarios in PLA.NET erscheint.
`Public Function CreateInstance() As IUpdater Implements IUpdater.CreateInstance`	Erzeugt eine Instanz der Importer-Routine.

4.2.4 IRVSzenarioUpdater

Für die automatisierte Befüllung eines RV-Szenarios in PLA.NT ist die Schnittstelle „IRVSzenarioUpdater“ zu implementieren:

```
Implements IRVSzenarioUpdater
```

Die Schnittstelle wird über das Kontextmenü eines bestehenden RV-Szenarios aufgerufen:



Mithilfe dieser Schnittstelle kann jede Position der verschiedenen RV-Knoten über eine externe Datei (bspw. einer Excel-Datei) befüllt und ggf. überschrieben werden. Das Erzeugen eines neuen RV-Szenarios ist nicht möglich.

Hinweis: Analog zum RV-Szenario Importer (siehe Kapitel 4.2.2) ist es aufgrund der umfangreichen Programmierung oft effizienter die Befüllung eines RV-Szenarios manuell vorzunehmen.

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	***Beschreibung***
`Public Function GetKey() As Guid Implements IUpdater.GetKey`	Liefert eine eindeutige ID (Guid) zurück. Hierfür muss vorab eine Guid definiert worden sein.
`Public Function GetName() As String Implements IUpdater.GetName`	Liefert den individuellen Namen zum Aufruf der Updater-Routine, welcher unter dem Kontextmenü-Eintrags eines RV-Szenarios in PLA.NET erscheint.
`Public Function CreateInstance() As IUpdater Implements IUpdater.CreateInstance`	Erzeugt eine Instanz der Updater-Routine.

4.3 Export

4.3.1 IResultQuery

Für eine benutzerdefinierte Ergebnisabfrage ist die Schnittstelle „IResultQuery“ zu implementieren:

```
Implements IResultQuery
```

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Function GetKey() As Guid`	Liefert eine eindeutige ID (Guid) zurück.
`Function GetName() As String`	Liefert den individuellen Namen zum Aufruf der Ergebnis-Routine.
`Function CreateInstance() As IResultQuery`	Erzeugt eine Instanz der Ergebnisabfrage.
`Sub Execute(Context As IResultQueryCon-text)`	Führt die Ergebnisabfrage durch. Mithilfe des Context-Objekts kann auf das Ergebnis zugegriffen werden.

4.3.2 IBatchResultQuery

Für eine benutzerdefinierte Batch-Ergebnisabfrage ist die Schnittstelle „IBatchResultQuery“ zu implementieren:

```
Implements IBatchResultQuery
```

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Function GetKey() As Guid`	Liefert eine eindeutige ID (Guid) zurück.
`Function GetName() As String`	Liefert den individuellen Namen zum Aufruf Ergebnis-Routine.
`Function CreateInstance() As IResultQuery`	Erzeugt eine Instanz der Ergebnisabfrage.
`Sub Execute(Context As IResultQueryCon-text)`	Führt die Ergebnisabfrage durch. Mithilfe des Context-Objekts kann auf die Ergebnisse zugegriffen werden.
`Sub ExecuteBatch(Contexts As IResultQuery-Context())`	Führt die Batch-Ergebnisabfrage durch. Mithilfe des Contexts-Objekts der Schnittstelle

Funktion / Sub	*Beschreibung*
	IResultQueryContext kann auf die Ergebnisse des Batchlaufes zugegriffen werden.

4.3.3 IPlanetResultQuery

Wird zum Öffnen einer ResultQuery für ein einzelnes Ergebnis oder der Teilberechnungen eines Batchlaufs verwendet:

```
Implements IPlanetResultQuery
```

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Function OpenResultQuery(OutputID As Integer) As IResultQueryContext`	Öffnet eine ResultQuery zum angegebenen Ergebnis.
`Function OpenResultQueryAsync(OutputID As Integer) As Task(Of IResultQueryContext)`	Öffnet eine ResultQuery zum angegebenen Ergebnis (asynchron).
`Function OpenBatchResultQuery(BatchID As Integer) As IEnumerable(Of IResultQueryContext)`	Öffnet zum angegebenen Batchlauf zu jeder Teilberechnung eine ResultQuery.
`Function OpenBatchResultQueryAsync(BatchID As Integer) As Task(Of IEnumerable(Of IResultQueryContext))`	Öffnet zum angegebenen Batchlauf zu jeder Teilberechnung eine ResultQuery (asynchron).

4.3.4 IResultQueryContext

Für einen Zugriff auf die Ergebnisdaten ist die Schnittstelle „IResultQueryContext“ zu implementieren:

```
Implements IResultQueryContext
```

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Sub Release()`	Gibt den Verweis auf die Ergebnisdaten frei.
`Function GetResultInfo() As IResultInfo`	Gibt einen Verweis auf die allgemeinen Ergebnisdaten zurück.
`Function GetMsgMgr() As IMsgMgr`	Gibt einen Verweis auf den Nachrichtenmanager von PLA.NET zurück.

Funktion / Sub	*Beschreibung*
`Function GetPivotQuerySource() As IPivot-QuerySource`	Gibt einen Verweis auf ein Zugriffsobjekt für den Pivotwürfel zurück.
`Function GetBilanz(KMS As Integer) As IBi-lanzElement()`	Gibt die Bilanz-Daten für ein angegebenes Szenario zurück, wobei KMS die Szenario Nummer ist.
`Function GetGuV(KMS As Integer) As IGuVElement()`	Gibt die GuV-Daten für ein angegebenes Szenario zurück.
`Function GetAssetportfolio(KMS As Integer, ProjYear As Integer) As IAssetPortfolio`	Gibt das Assetportfolio zu einem angegebenen Szenario und Zeitpunkt zurück.
`Function GetKapitalanlage(KMS As Integer) As IKapitalanlageMantelElement()`	Gibt die Kapitalanlageentwicklung für ein angegebenes Szenario zurück.
`Function GetLeistungsspektrumLeben(KMS As Integer, Ordnung As Integer) As LifeRe-sult()`	Gibt die Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetLeistungsspektrumLeben(KMS As Integer, Ordnung As Integer, Bestands-gruppe As Integer) As LifeResult()`	Gibt die Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetLeistungsspektrumLebenMtl(KMS As Integer, Ordnung As Integer) As LifeRe-sult()`	Gibt die monatlichen Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetLeistungsspektrumLebenMtl(KMS As Integer, Ordnung As Integer, Bestands-gruppe As Integer) As LifeResult()`	Gibt die monatlichen Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetLeistungsspektrumKranken(KMS As Integer, Ordnung As Integer) As Health-Result()`	Gibt die Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetLeistungsspektrumKranken(KMS As Integer, Ordnung As Integer, Bestands-gruppe As Integer) As HealthResult()`	Gibt die Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetErweitertesLeistungsspektrum-ImJahr(Jahr As Integer, Ordnung As Inte-ger) As IErweitertesLeistungsspektrumEle-ment()`	Gibt die jährlichen erweiterten Leistungsspektrum-Daten für ein angegebenes Projektionsjahr zurück.
`Function GetErweitertesLeistungsspektrum-ImJahr(Jahr As Integer, Ordnung As Inte-ger, Bestandsgruppe As Integer) As IErwei-tertesLeistungsspektrumElement()`	Gibt die jährlichen erweiterten Leistungsspektrum-Daten für ein angegebenes Projektionsjahr zurück.
`Function GetErweitertesLeistungsspekt-rum(KMS As Integer, Ordnung As Integer) As IErweitertesLeistungsspektrumElement()`	Gibt die jährlichen erweiterten Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetErweitertesLeistungsspekt-rum(KMS As Integer, Ordnung As Integer, Bestandsgruppe As Integer) As IErweiter-tesLeistungsspektrumElement()`	Gibt die jährlichen erweiterten Leistungsspektrum-Daten für ein angegebenes Szenario zurück.
`Function GetErweitertesLeistungs-spektrumMtl(KMS As Integer, Ordnung As In-teger) As IErweitertesLeistungsspektrum-Element()`	Gibt die monatlichen erweiterten Leistungsspekt-rum-Daten für ein angegebenes Szenario zurück.
`Function GetErweitertesLeistungs-spektrumMtl(KMS As Integer, Ordnung As In-teger, Bestandsgruppe As Integer) As IEr-weitertesLeistungsspektrumElement()`	Gibt die monatlichen erweiterten Leistungsspekt-rum-Daten für ein angegebenes Szenario zurück.
`Function GetInterneRechnungslegungLife(KMS As Integer, Ordnung As Integer) As IInter-neRechnungslegungLife()`	Gibt die internen Rechnungslegungsdaten für ein angegebenes Szenario zurück.

Funktion / Sub	*Beschreibung*
`Function GetInterneRechnungslegungLife(KMS As Integer, Ordnung As Integer, Bestandsgruppe As Integer) As IInterneRechnungslegungLife()`	Gibt die internen Rechnungslegungsdaten für ein angegebenes Szenario zurück.
`Function GetInterneRechnungslegungHealth(KMS As Integer, Ordnung As Integer) As IInterneRechnungslegungHealth()`	Gibt die internen Rechnungslegungsdaten für ein angegebenes Szenario zurück.
`Function GetInterneRechnungslegungHealth(KMS As Integer, Ordnung As Integer, Bestandsgruppe As Integer) As IInterneRechnungslegungHealth()`	Gibt die internen Rechnungslegungsdaten für ein angegebenes Szenario zurück.
`Function GetKennzahlen(KMS As Integer) As IQuantitativeErgebnisseElement()`	Gibt die Kennzahlen-Daten für ein angegebenes Szenario zurück.
`Function GetCashflowSicht(KMS As Integer) As ICashflowSicht()`	Gibt die Cashflow-Sicht für ein angegebenes Szenario zurück.
`Function GetRfBEntwicklung(KMS As Integer) As IRfBEntwicklungElement()`	Gibt die RfB-Entwicklung für ein angegebenes Szenario zurück.
`Function GetRfBEntwicklung(KMS As Integer, Bestandsgruppe As Integer) As IRfBEntwicklungElement()`	Gibt die RfB-Entwicklung für ein angegebenes Szenario zurück.
`Function GetHealthBeitragsbewegung(KMS As Integer) As IHealthBeitragsbewegungElement()`	Gibt die Beitragsbewegung in der Krankenversicherung für ein angegebenes Szenario zurück.
`Function GetHealthBeitragsbewegung(KMS As Integer, Bestandsgruppe As Integer) As IHealthBeitragsbewegungElement()`	Gibt die Beitragsbewegung in der Krankenversicherung für ein angegebenes Szenario zurück.
`Function GetHealthLeistungsbewegung(KMS As Integer) As IHealthLeistungsbewegungElement()`	Gibt die Leistungsbewegung in der Krankenversicherung für ein angegebenes Szenario zurück.
`Function GetHealthLeistungsbewegung(KMS As Integer, Bestandsgruppe As Integer) As IHealthLeistungsbewegungElement()`	Gibt die Leistungsbewegung in der Krankenversicherung für ein angegebenes Szenario zurück.
`Function GetKompositSpartenInfos() As IKompositSpartenInfo()`	Gibt die Komposit-Sparteninformationen des Ergebnisses zurück.
`Function GetKompositRVInfos() As IKompositRVInfo()`	Gibt die Komposit-Rückversicherungsvertragsinformationen des Ergebnisses zurück.
`Function GetKompositRVDefs() As IKompositRVDef()`	Gibt die Komposit-Rückversichererinformationen des Ergebnisses zurück.
`Function GetKompositVtR(KMS As Integer) As IKompositVtR`	Gibt eine Zugriffsschnittstelle auf die verschiedenen versicherungstechnischen Rechnungen des Ergebnisses zurück.
`Function GetKompositAbwicklung(KMS As Integer) As IKompositAbwicklung`	Gibt eine Zugriffsschnittstelle auf die verschiedenen Abwicklungsdaten des Ergebnisses zurück.
`Function GetKompositErwLeistungsspektrum(KMS As Integer) As IKompositELS`	Gibt eine Zugriffsschnittstelle auf das erweiterte Leistungsspektrum des Ergebnisses zurück.
`Function GetKompositSCR(KMS As Integer, Art As Integer) As IKompositSCRData()`	Gibt die SCR-Daten in der Schaden-/Unfallversicherung für ein angegebenes Szenario zurück.
`Function GetUserResult(UserResultType As Guid, KMS As Integer) As IUserResult`	Gibt ein benutzerdefiniertes Ergebnis vom angegebenen Typ und Szenario zurück.

Funktion / Sub	*Beschreibung*
`Function GetUserFigures(KMS As Integer) As IUserFigureMgr()`	Gibt die benutzerdefinierten Kennzahlen für ein angegebenes Szenario zurück.

4.4 Managementregeln / Hooks

Mittels benutzerdefinierten Managementregeln können eigene Managementregeln in PLA.NET programmiert werden. Dabei muss aus technischer Sicht zwischen den zwei Komponenten „Managementregel“ und „Hook“ grundlegend unterschieden werden.

4.4.1 Allgemeines

Hook
Hooks sind aus technischer Sicht die eigentliche Managementregel (also der Algorithmus / die Berechnungsvorschrift für den anzupassenden Sachverhalt) und werden immer über eine Schnittstelle (bspw. „IAdjustVtRHook“, siehe Kapitel 4.4.2) eingebunden.

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Public Function GetKey() As Guid Implements IHook.GetKey`	Liefert eine eindeutige ID (Guid) zurück. Hierfür muss vorab eine Guid definiert worden sein.
`Public Function GetName() As String Implements IHook.GetName`	Gibt den individuellen Namen des Hooks zurück.
`Public Function GetPriority() As Integer Implements IHook.GetPriority`	Gibt die Priorität des Hooks zurück.

<u>Managementregel</u>
Eine Managementregel dient aus technischer Sicht zur Verknüpfung des Hooks mit der Oberfläche im Bereich Managementregeln in PLA.NET. Dies erlaubt es, freie Eingaben pro MMR zu definieren und speichern zu können. Managementregeln werden mittels des Konzepts der Vererbung in PLA.NET eingebunden (`Inherits Managementregel`).

Folgende Funktionen müssen dabei standardmäßig implementiert werden:

Funktion / Sub	*Beschreibung*
`Public Overrides Function MMR_Name() As String`	Name der Managementregel, welcher an der Oberfläche im Bereich „Managementregeln“ ausgewiesen werden soll.
`Public Overrides Function MMR_Nr() As Guid`	Eindeutige Identifizierung der MMR durch Guid.

Funktion / Sub	*Beschreibung*
`Public Overrides Function SortOrder() As Integer`	Reihenfolge der MMR
`Public Overrides Function ZugeordnetZu() As Art`	Legt den Bereich innerhalb der Managementregeln fest. Folgende Ausprägungen können als Art gewählt werden: - AktivRegel = 1 - PassivRegel = 2 - SonstigeParameter = 3
`Public Overrides Function IsLifeMMR() As Boolean`	Funktion mit welcher angegeben werden kann, ob die MMR zur Sparte Leben gehört oder nicht.
`Public Overrides Function IsHealthMMR() As Boolean`	Funktion mit welcher angegeben werden kann, ob die MMR zur Sparte Kranken gehört oder nicht.
`Public Overrides Function isNonLifeMMR() As Boolean`	Funktion mit welcher angegeben werden kann, ob die MMR zur Sparte Schaden-/ Unfall gehört oder nicht.
`Public Overrides Function NecessaryForProfittest() As Boolean`	Funktion mit welcher angegeben werden kann, ob die MMR für den Profittest notwendig ist (für Schaden-/ Unfall nicht relevant – auf False zu setzen).
`Public Overrides Function NecessaryForSimpleAktiva() As Boolean`	Funktion mit welcher angegeben werden kann, ob die MMR für den Profittest notwendig ist (für Schaden-/ Unfall nicht relevant – auf False zu setzen).
`Public Overrides Function NecessaryForAssetOnly() As Boolean`	Für Schaden-/ Unfall nicht relevant – auf False zu setzen.

4.4.2 IAdujstVtR

Mittels der Schnittstelle „IAdjustVtRHook“ kann die versicherungstechnische Rechnung (VtR) Brutto einer PLA.NET-Berechnung pro Pfad angepasst werden. Damit kann bspw. die HGB-Reserve abhängig von einer Solvency-II Reserve bestimmt oder HGB-Reserven bei extremen Ereignissen aufgelöst werden.

Es ist darauf hinzuweisen, dass zum jeweiligen Berechnungszeitpunkt nur die aktuellen Bruttowerte zur Verfügung stehen.

Der Hook wird wie folgt eingebunden:

```
Implements IAdjustVtRHook
```

Dabei müssen neben den allgemeinen Funktionen (siehe Kapitel 4.4.1) folgende zusätzliche Funktionen / Subs implementiert werden:

Funktion / Sub	*Beschreibung*
`Public Function CreateInstance() As IAdjustVtRHook Implements IAdjustVtRHook.CreateInstance`	Gibt eine neue Instanz des jeweiligen Hooks (hier „IAdjustVtRHook“) zurück.

Funktion / Sub	*Beschreibung*

`Public Sub AdjustVtR(`
` - Context As IHookContext,`
` - KSID As Integer,`
` - AbwDaten As PlanetAbwicklung.IAbwDaten,`
` - VtR As VersTechRechnung,`
` - VtrVJ As VersTechRechnung,`
` - BruttoAbgBasis As RVAbgabeBasis)`
`Implements IAdjustVtRHook.AdjustVtR`
	Sub, welche die Anpassung der VtR durchführt.

Die genaue Berechnungsvorschrift wie bestimmte Positionen der VtR angepasst werden obliegt dem Anwender.

4.4.3 IDividendenHook

Mittels des IDividendenHooks können Dividenden ausgeschüttet werden. Es empfiehlt sich diesen Hook mit einer MMR zu verknüpfen, um geplante Ausschüttungen an der Oberfläche vorgebbar machen zu können. Wahlweise können auch komplexere Ausschüttungsmechanismen programmiert werden.

Der Hook wird wie folgt eingebunden:

```
Implements IAdjustVtRHook
```

Dabei müssen neben den allgemeinen Funktionen (siehe Bereich Managementregeln / Allgemeines) folgende zusätzliche Funktionen / Subs implementiert werden:

Funktion / Sub	*Beschreibung*
`Public Function CreateInstance() As IDividendenHook Implements IDividendenHook.CreateInstance`	Gibt eine neue Instanz des jeweiligen Hooks (hier „IDividendenHook“) zurück.

`Public Function GetDividende(`
` - Context As IHookContext,`
` - ProjYear As Integer)`
`As Double Implements IDividendenHook.GetDividende`
	Liefert die Dividende des Hooks an den PLA.NET-Kern zurück. In der Regel wird über den Context eine entsprechende Dividende ausgelesen, welche über eine Managementregel vorgegeben wurde.

<u>Fachliches</u>

Der Dividendenhook wirkt bilanzverkürzend. Der jeweilige Betrag, welcher die Funktion GetDividende() (siehe obige Tabelle) zurückgibt, kürzt das Eigenkapitel des folgenden Projektionsjahres. Anders formuliert: Es wird das Eigenkapital um den genannten Betrag gekürzt – sichtbar in der Bilanz (Pos. Eigenkapital) im darauffolgenden Projektionsjahr.

4.5 Tarifklassenprogrammierung

Über eine Schnittstelle ist es möglich eine eigene Tarifklasse zu definieren und so die Berechnung der detaillierten Beitragsvektoren individuell zu gestalten, sowie individuell Komponenten des Bestands zu definieren. Für die Tarifklasse sind folgende Schnittstellen nötig:

4.5.1 INonlifeTariffCalculator

Die Schnittstelle INonlifeTariffCalculator repräsentiert ein Berechnungsobjekt für einen Tarif. Dieses Berechnungsobjekt implementiert die Berechnungslogik für die Berechnung eines Modelpoints pro Projektionsjahr.

Propertys / Sub	***Beschreibung***
`Property Pol As BasePolicy`	Bestandsatz des zu berechnenden Modelpoints Hier können alle Komponenten des Bestands definiert werden.
`Property Carry As CarryOver`	Übertrag des zu berechnenden Modelpoints.
`Property LS As NonlifeResult`	Ergebnisobjekt des aktuellen Jahres
`Sub Prepare(ProjYear As Integer, DueDate As Date, InputParameter As NonlifeInputParameter)`	Berechnung des Aufsetzpunktes vor Projektionsbeginn. Parameter: aktuelles Projektionsjahr, Stichtag (DueDate) und Schaden-/Unfallspezifische Eingabeparameter (Inputparameter)
`Sub Calculate(ProjYear As Integer, DueDate As Date, InputParameter As NonlifeInputParameter)`	Berechnung der Leistungsspektrumsgröße Parameter: aktuelles Projektionsjahr, Stichtag (DueDate) und Schaden-/Unfallspezifische Eingabeparameter (Inputparameter)

4.5.2 INonLifeTariff

Propertys / Sub	***Beschreibung***
`ReadOnly Property Ab As ActuarialBasis`	Das dem Tarif zugeordnete Rechnungsgrundlagen-Objekt
`Function CreateCalculator() As INonlifeTariffCalculator`	Erzeugt ein zum Tarif passendes TariffCalculator-Objekt

4.5.3 ITariffClass

Beschreibt eine Tarifklasse.

Propertys / Sub	*Beschreibung*
`Sub Init(MsgMgr As IMsgMgr, MMRs As IManagementregeln)`	Initialisiert die Tarifklasse; Übergabe von Zugriffsschnittstellen auf Benachrichtigung und Managementregeln Parameter: MsgMgr: Benachrichtigung-Zugriff, MMRs: Managementregeln-Zugriff
`Sub Reset()`	Zurücksetzen des internen Status der Tarifklasse
`Function GetClassName() As String`	Name der Tarifklasse (Max. Länge 20 Zeichen)
`Function GetTariffClassType() As Versicherungsarten`	Gibt die Sparten der Tarifklasse zurück
`Function GetTariffClassOptions() As TariffClassOptions`	Gibt eine oder mehrere Optionen zur Charakterisierung der Tarifklasse zurück
`Function CreateSettings() As TariffClassSettings`	Wird nicht benötigt
`Function GetSettingsType() As Type`	Wird nicht benötigt
`Function CreateTariff(Stngs As TariffClassSettings, ProjDuration As Integer, Gender As String, Ab As ActuarialBasis) As ITariff`	Erzeugt einen neuen zur Tarifklasse gehörenden Tarif für das angegebene Geschlecht
`Function GetGenders() As String()`	Wird nicht benötigt, Rückgabe-String: `"-"`
`Function CreateActuarialBasis(TabMgr As ITableMgr, DeclMgr As IDeclarationMgr) As ActuarialBasis`	Erzeugt ein neues Rechnungsgrundlagen-Objekt für einen Tarif, der zur Tarifklasse gehört
`Function GetActuarialBasisType() As Type`	Gibt den Typ des Rechnungsgrundlagen-Objekts der Tarifklasse zurück
`Function CreatePolicy() As BasePolicy`	Erzeugt ein neues Modelpoint-Objekt
`Function GetPolicyType() As Type`	Gibt den Typ der Modelpoint-Objekte zurück
`Function CreateCarryOver() As CarryOver`	Gibt ein neues Übertrag-Objekt zurück
`Public Enum TariffClassOptions`	Tarifklassenoptionen, wird nicht benötigt

4.6 Sonstiges

4.6.1 IRueckversicherungUI

Mithilfe der Schnittstelle „IRueckversicherungUI“ kann ein benutzerindividueller Rückversicherungsknoten erstellte werden, welcher im RV-Szenario dann zur Verfügung steht.

Propertys / Sub	*Beschreibung*
`Function GetKey() As Guid`	Eindeutiger Schlüssel der Rückversicherung
`Function GetShortKey() As Integer`	Kurzer Schlüssel als Integer der Rückversicherung
`Function CreateInstance() As IRueckversicherungUI`	Funktion zum Erstellen der Oberfläche des Rückversicherungsknotens
`Sub InitUI(Context As IRueckversicherungUIContext)`	Befüllen des Rückversicherungsknotens

4.6.2 IRegisterRueckversicherung

Mithilfe der Schnittstelle „IRegisterRueckversicherung“ kann eine benutzerindividuelle Rückversicherung angelegt werden.

Propertys / Sub	*Beschreibung*
`Function GetKey() As Guid`	Eindeutiger Schlüssel der Rückversicherung
`Function GetShortKey() As Integer`	Kurzer Schlüssel als Integer der Rückversicherung
`Function GetName() As String`	Gibt den Namen der Rückversicherung zurück als String
`Function GetRVInfo() As RVFactoryInfo`	Gibt diverse Informationen zur Rückversicherung zurück

4.6.3 IRegisterSUVCarryProperties

Diese Schnittstelle wird für individuelle Rückversicherungslösungen benötigt.

Propertys / Sub	*Beschreibung*
`Sub Register(CDM As CarryDisplayMgr)`	Register bestimmter Parameter in CarryDisplayMgr, welche technische Informationen enthält

4.6.4 ISUVCustomParameter

Die Schnittstelle wird für individuelle Rückversicherungslösungen benötigt. Mit „ISUVCustomParameter“ wird das Laden und Speichern der Rückversicherungsdaten gesteuert.

Propertys / Sub	*Beschreibung*
`Sub LoadFromReader(reader As XmlReader, KSID As Integer, Jahr As Integer)`	Liest die Daten der Rückversicherung aus der XML-Datei
`Sub SaveToWriter(writer As XmlWriter, KSID As Integer, Jahr As Integer)`	Schreibt die Daten der Rückversicherung in die XML-Datei
`Function KSIDCount() As Integer`	Gibt die Anzahl der KSIDs zurück
`Function GetKSID(i As Integer) As Integer`	Gibt eine bestimme KSID zurück

4.6.5 IUserEinzelschadenRunOff

Mithilfe dieser Schnittstelle lassen sich benutzerdefinierte Einzelschaden Run-Off Methoden definieren, welche in PLA.NET in der Bruttosparte im Reiter Einzelschäden oder Rentenschäden angewendet werden können.

Propertys / Sub	*Beschreibung*
`Function GetID() As Integer`	Gibt die ID der Run-Off Methode zurück.
`Function GetName() As String`	Gibt den Namen der Run-Off Methode zurück.
`Function CreateInstance() As IUserEinzelschadenRunOff`	Erstellt eine Instanz der ausgewählten Einzel-schaden-RunOff-Methode.
`Sub RunOff(context As IUserEinzelschaden-RunOffContext, einzelschaden As Einzel-schaden, firstFJ As Integer)`	Ausführen der Run-Off Methode. Hier ist die konkrete Methodik zu implementieren.

4.6.6 IUserEinzelschadenRunOffContext

Diese Schnittstelle ermöglicht den Zugriff auf die Abwicklungsfaktoren und wird als Parameter der Methode *IUserEinzelschadenRunOff.RunOff* übergeben.

Folgende Zugriffsfunktionen stehen zur Verfügung:

Propertys / Sub	***Beschreibung***
`Function GetBasisAbwicklungsfaktoren() As Jahresvektor(Of Abwicklungsfaktor)`	Gibt die Basis-Abwicklungsfaktoren zurück.
`Function GetGrossAbwicklungsfaktoren() As Jahresvektor(Of Abwicklungsfaktor)`	Gibt die Großschäden-Abwicklungsfaktoren zurück.
`Function GetGefahrAbwicklungsfaktoren() As Jahresvektor(Of Abwicklungsfaktor)`	Gibt die Kumulereignis-Abwicklungsfaktoren anhand der Ereignisgefahr zurück.
`Function GetZusatzAbwicklungsfaktoren(AbwID As Integer?) As Jahresvektor(Of Abwicklungsfaktor)`	Ermittelt die Abwicklungsfaktoren anhand der Zusatz-Abwicklungsfaktoren mit der angegebenen ID.
`Function GetZusatzAbwicklungsfaktoren(beschreibung As String) As Jahresvektor(Of Abwicklungsfaktor)`	Ermittelt die Abwicklungsfaktoren anhand der Zusatz-Abwicklungsfaktoren mit der angegebenen Beschreibung